

Exercice 1 : Implémentation d'une pile avec la récursivité**Objectif :**

Créer une pile dynamique et utiliser la récursivité pour :

1. Ajouter des éléments dans la pile (fonction push).
2. Afficher les éléments de la pile dans l'ordre inverse (fonction display).

Instructions :

1. Implémentez une pile en utilisant une structure Node avec deux champs :
 - data pour stocker la valeur.
 - next pour pointer vers le nœud suivant.
2. Écrivez une fonction récursive display(Node* top) pour afficher tous les éléments.
3. Écrivez une fonction push(Node** top, int value) pour ajouter un élément.

Structure de départ :

```
1. #include <stdio.h>
2. #include <stdlib.h>
3.
4. typedef struct Node {
5.     int data;
6.     struct Node* next;
7. } Node;
8.
9. // Déclarez vos fonctions ici
10. void push(Node** top, int value);
11. void display(Node* top);
12.
13. int main() {
14.     Node* stack = NULL; // Pile vide
15.
16.     // Ajout d'éléments
17.     push(&stack, 1);
18.     push(&stack, 2);
19.     push(&stack, 3);
20.
21.     // Affichage de la pile
22.     printf("Contenu de la pile :\n");
23.     display(stack);
24.
25.     return 0;
26. }
27.
```

Exercice 2 : Recherche dans une liste chaînée avec la récursivité

Objectif :

Créer une liste chaînée et utiliser la récursivité pour chercher un élément spécifique dans la liste.

Instructions :

1. Implémentez une structure Node pour la liste chaînée avec deux champs :
 - o data pour la valeur.
 - o next pour pointer vers le nœud suivant.
2. Implémentez une fonction récursive search(Node* head, int value) :
 - o Retourne 1 (vrai) si la valeur est trouvée.
 - o Retourne 0 (faux) sinon.

Exemple attendu :

Liste chaînée : 1 → 2 → 3 → 4 → NULL

- Recherche de 3 : Résultat = Trouvé.
- Recherche de 5 : Résultat = Non trouvé.

Structure de départ :

```
1. #include <stdio.h>
2. #include <stdlib.h>
3.
4. typedef struct Node {
5.     int data;
6.     struct Node* next;
7. } Node;
8.
9. // Déclarez vos fonctions ici
10. Node* createNode(int value);
11. void append(Node** head, int value); Ajout à la fin de liste
12. int search(Node* head, int value);
13.
14. int main() {
15.     Node* list = NULL; // Liste vide
16.
17.     // Ajout d'éléments
18.     append(&list, 1);
19.     append(&list, 2);
20.     append(&list, 3);
21.     append(&list, 4);
22.
23.     // Recherche récursive
24.     int value = 3;
25.     if (search(list, value))
26.         printf("Valeur %d trouvée.\n", value);
27.     else
28.         printf("Valeur %d non trouvée.\n", value);
29.
30.     value = 5;
31.     if (search(list, value))
32.         printf("Valeur %d trouvée.\n", value);
33.     else
34.         printf("Valeur %d non trouvée.\n", value);
35.
36.     return 0;
37. }
38.
```